Evaluación V (Semana 10) — Demo Day de Avance Funcional (Primer CRUD), GitFlow/Kanban y Valoración de Costos TI

Auditoría integral en dos instancias obligatorias: (1) Demostración ejecutiva en aula (5 a 7 min) del flujo del módulo y su costeo, y (2) Envío formal del repositorio GitHub para clonación y revisión técnica asíncrona de avances reales por el docente (Frontend dinámico consumiendo APIs RESTful desacopladas con persistencia activa en Base de Datos), trazabilidad colaborativa en **Tableros Ágiles (GitHub Projects, Trello, Asana, Monday.com, Jira)** y **GitFlow**, y sustentación del **Informe Técnico-Económico de Valoración del Software** modelado bajo la Metodología Oficial de 7 Pasos y Banda Absoluta de Precios.

DOCENTE TITULAR	MODALIDAD	CORTE ACADÉMICO	PONDERACIÓN SUMATIVA
Prof. Eduardo Nieves	Auditoría de Código y Demo en Aula	Segundo Corte • Lapso 2-2026	15% Neto (Escala 100%)

1 Estructura Didáctica de la Sesión de Auditoría (Metodología de Educación Popular)

1. MOMENTO ANTES (20 MIN)

Recepción de Repositorios y Conciliación: Entrega física/digital del Informe de Costos, verificación de enlaces GitHub (Monorepo o repos independientes frontend/backend; si es privado, invitación a `zedmous` / `zedmous@gmail.com`), tablero ágil y verificación de commits en `develop`.

2. MOMENTO DURANTE (115 MIN)

Defensa Ágil por Célula (5 a 7 min c/u): Demostración ejecutiva y concisa en aula. Cada célula defiende: (1) **Flujo del Módulo (3-4 min):** ejecución del flujo funcional en vivo (interfaz conectada a API y persistencia en DB); y (2) **Costo del Módulo Entregado (2-3 min):** sustentación del valor económico del bloque construido según horas reales de la tarjeta y los 7 pasos.

3. MOMENTO DESPUÉS (45 MIN)

Balance de Deuda Técnica y Calificación: Evaluación sumativa inmediata con rúbrica, análisis de consistencia contable del software desarrollado y retroalimentación de arquitectura para el cierre de semestre (Tercer Corte).

2 Competencias Profesionales Exigidas y Entregables Obligatorios

Competencia Auditada	Indicador de Desempeño Observable	Evidencia Entregable en Aula
Construcción e Integración Full-Stack	Interconecta interfaces de usuario reactivas con APIs RESTful y persistencia real en motores SQL, gestionando estados asíncronos y validaciones.	• Software ejecutándose en vivo en navegador. • Base de datos con datos relacionales persistidos. • Endpoints probados con Postman / ThunderClient.
Gobierno Ágil con Git y Kanban	Aplica la estrategia GitFlow rigurosamente (ramas por feature, PRs auditados hacia `develop`) reflejando el progreso y las horas en el tablero ágil elegido (GitHub Projects, Trello, Asana, Monday o Jira).	• Tablero Kanban activo con 4 columnas normadas. • Pull Requests mergeados con historial limpio. • Registro estricto de horas reales por ticket.
Ingeniería Económica de Software	Modela con exactitud matemática el costo de manufactura de software y fija precios blindados mediante la metodología de 7 pasos.	• **Informe de Valoración de Software** impreso. • **Matriz Espejo de Conciliación** Horas-Tablero. • Banda Absoluta de Precios calculada.

Regla de Tolerancia Cero #1: Prohibición de Datos Simulados

Se penalizará con **0 puntos en la dimensión de Backend** a proyectos que presenten tablas con datos fijos en HTML o arreglos estáticos en JS. La data debe originarse desde la base de datos.

Regla de Tolerancia Cero #2: Descuadre Horas vs. Costos

Si las horas sumadas en los tickets de la columna *Terminado* no coinciden exactamente con las horas base del Paso 3 del Informe Económico, el informe se anula por **inconsistencia contable**.

3 Especificaciones Técnicas Obligatorias para la Capa de Cliente (Frontend)

Habiendo aprobado el Sistema de Diseño UI/UX en la evaluación previa de este Segundo Corte (Semana 8: Maquetación y Modelo de Datos), las vistas deben evolucionar de maquetas estáticas a componentes dinámicos con renderizado reactivo y consumo de red:

1. Consumo Asíncrono de APIs

- Prohibición de arrays de prueba `const data = [...]` en el cliente.
- Implementación de peticiones mediante `fetch()` o instancias de `axios` centralizadas.
- Uso correcto de `async/await` con captura de excepciones en bloques `try/catch`.

2. Los 3 Estados de Interfaz (UI)

- **Cargando (Loading):** Despliegue de spinner o skeleton screen mientras la promesa HTTP está pendiente.
- **Éxito (Success):** Modales, toasts o feedback visual inmediato al crear, editar o eliminar registros.
- **Error (Error Handling):** Notificaciones legibles al usuario ante fallos 400, 404 o 500 del servidor.

3. Formularios y Validación

- Validación en tiempo de captura (campos requeridos, expresiones regulares, formatos numéricos).
- Deshabilitación de botones de acción (`disabled`) durante el dispatch para evitar duplicados.
- Limpieza de inputs y refresco automático de tablas tras mutaciones exitosas.

4 Especificaciones Técnicas para la Capa de Servidor y Persistencia (Backend & DB)

Arquitectura de Software en Capas (Backend)

El servidor (Node.js/Express, Python/FastAPI, PHP/Laravel, etc.) debe mantener separación modular estricta:

- **Capa de Enrutamiento (Routes):** Definición de endpoints REST puros asociados a verbos HTTP estandarizados.
- **Capa de Controladores (Controllers):** Recepción del `Request`, orquestación de llamadas y respuesta en `Response`.
- **Capa de Lógica/Servicios (Services):** Reglas de negocio del sistema (cálculo de totales, validación de stock, etc.).
- **Manejo Global de Errores:** Middleware centralizado que atrape excepciones sin tumbar el proceso del servidor.

Persistencia y Modelo de Datos Relacional (DB)

Se auditará directamente el motor relacional (PostgreSQL / MySQL / SQLite):

- **Conexión Activa y Segura:** Credenciales parametrizadas mediante variables de entorno (`.env`).
- **Integridad Referencial:** Tablas con Claves Primarias (`PK`), Claves Foráneas (`FK`) y restricciones `NOT NULL` / `UNIQUE`.
- **Trazabilidad DDL:** Existencia de archivos de migración o script SQL estructurado en la carpeta `/database` del repositorio.
- **Operaciones CRUD:** Al menos un módulo clave con Create, Read, Update y Delete/Soft Delete 100% operativo.

5 Tabla de Verificación de Endpoints del Módulo Evaluado

Verbo HTTP	Patrón de Ruta REST	Responsabilidad Operativa	Código de Estado Esperado
GET	`/api/{modulo}`	Listado paginado o total de entidades activas.	`200 OK`
GET	`/api/{modulo}/:id`	Consulta detallada de una entidad específica.	`200 OK` / `404 Not Found`
POST	`/api/{modulo}`	Inserción de nuevo registro con validación de payload.	`201 Created` / `400 Bad Request`
PUT / PATCH	`/api/{modulo}/:id`	Actualización total o parcial de campos permitidos.	`200 OK` / `422 Unprocessable`
DELETE	`/api/{modulo}/:id`	Eliminación física o inactivación lógica de estado.	`200 OK` / `204 No Content`

Doble Instancia de Evaluación: Demostración en Aula (5 a 7 min) vs. Revisión en Repositorio Clonado:

• En Clase (Defensa Presencial):

Presentación breve y ejecutiva (5 a 7 min). El equipo solo debe mostrar *parte del flujo del sistema* corriendo en vivo (crear/consultar datos en la UI con persistencia real en DB) y defender *cuánto vale el módulo entregado* según las horas reales de su tarjeta y los 7 pasos.

• En GitHub (Auditoría Asíncrona):

Una cosa es mostrar el software en clase y otra enviar el proyecto para revisión profunda. La entrega formal en GitHub es obligatoria (invitación a `@zedmous` / `zedmous@gmail.com`). El docente

clonará el proyecto para auditar el código fuente, verificar avances reales, inspeccionar commits por integrante y constatar que lo mostrado en aula exista físicamente en el repositorio

.

6 Ciclo de Vida de una Tarea: Las 4 Columnas del Tablero Ágil (GitHub Projects, Trello, Asana, Monday, Jira)

El tablero Kanban (gestionado en **GitHub Projects, Trello, Monday.com, Asana, ClickUp o Jira**) no es un adorno visual; es la bitácora legal del proyecto de software. Cada célula tiene plena libertad de elegir la herramienta que prefiera, siempre que respete las 4 columnas normadas y la trazabilidad de Git:

1. POR HACER — BACKLOG

Requerimiento Asignado
Issue creado con ID formal, criterios de aceptación y horas estimadas.

Condición: Tarea priorizada. Aún no se ha creado ninguna rama de desarrollo.

2. EN PROGRESO — WIP

Desarrollo Activo
El programador asignado está codificando la funcionalidad.

Condición: Rama creada desde `develop`: `feature/<num_ticket>-<titulo>` (ej: `feature/102-crud-facturas`).

3. EN REVISIÓN — PR OPEN

Auditoría de Código
Código listo para ser inspeccionado por sus compañeros.

Condición: Pull Request (PR) abierto desde `feature/<num_ticket>-<titulo>` hacia `develop`.

4. TERMINADO — MERGED

Funcionalidad Lista
Módulo probado, aprobado y listo para demostración.

Condición: PR aprobado y **mergeado** en `develop` (y release merge a `master`). Horas registradas.

7 El Requisito Clave: Registro Obligatorio de Horas Trabajadas por Ticket

En la industria del software y según la Guía Oficial de Costeo del Ing. Eduardo Nieves, la unidad atómica de costo es la **Hora Hombre por Tarea / Card / Ticket**. Por tanto, cada Issue que avance a *Terminado* debe contener obligatoriamente su registro de tiempo real:

¿Por qué registrar las horas en el ticket?

- **Evidencia de Esfuerzo Real:** Permite contrastar la complejidad estimada contra el tiempo real que tomó escribir el código, depurar bugs y hacer pruebas.
- **Auditoría de Trabajo en Equipo:** Demuestra si todos los miembros aportaron equitativamente o si hubo sobrecarga en un solo integrante.
- **Base Multiplicadora del Costo:** El total de horas reales es la cifra que se ingresará en el **Paso 3 (Mano de Obra)** del informe económico.

Plantilla Oficial de Tarjeta / Ticket (GitHub, Trello, Asana, Monday)

```
### 📌 Tarea: TASK-102 - CRUD Completo Endpoint Facturación
**Módulo:** Facturación y Ventas • **Responsable:** @estudiante_dev (Full-Stack)
**Git Lead Revisor:** @lider_tecnico (Code Review & Merge)

#### 📝 Descripción Técnica
Interfaz reactiva y controlador CRUD con validación DTO y persistencia relacional.

#### ⏱️ Trazabilidad de Tiempo Laboral
- **Horas Estimadas:** 8.0 h • **Horas Reales Invertidas:** 10.5 h (Desvío: ajuste FK)

#### 🔗 Evidencia GitFlow
- **Rama:** `feature/102-crud-facturacion` (desde `develop`)
- **Pull Request:** PR #14 (Aprobado y mergeado a `develop` por Git Lead)
```

8 Estándar Oficial de Ramas GitFlow: master, develop, feature/<numero_ticket>-<titulo>

Rama Obligatoria	Propósito en el Proyecto	Reglas Estrictas de Nomenclatura y Auditoría
`master` (o `main`)	Producción / Entrega Final	Código estable auditado. **Prohibido hacer commits directos**. Solo recibe merges desde `develop` mediante PR aprobado al cierre de la evaluación.
`develop`	Integración del Sprint	Rama base colectiva de trabajo diario. Todas las ramas funcionales nacen de aquí y se reintegran mediante PRs revisados por el Git Lead.
`feature/<num>-<titulo>`	Módulo / Tarea Asignada	**Formato estricto:** prefijo `feature/` + número de ticket + título descriptivo en kebab-case. *Ej:* `feature/101-crud-clientes`, `feature/102-crud-facturacion`, `feature/103-autenticacion-jwt`.
`hotfix/<num>-<titulo>`	Corrección de Emergencia	Rama auxiliar para bugs bloqueantes o caídas en la demo en vivo. *Ej:* `hotfix/104-fix-fk-facturas`. Nace de master/develop y se mergea de inmediato.

Arquitectura GitHub y Revisión de Avances:
Se acepta **Monorepo** o **Repositorios Separados**. Repos **privados** requieren invitación a `@zedmous` / `zedmous@gmail.com`. El docente clonará el código para auditar el avance técnico real, commits de cada miembro y consistencia con lo mostrado en el aula.

Full-Stack, Git Lead y Sanción Directa:
Los 5 integrantes desarrollan como **Full-Stack**; el **Git Lead** autoriza merges a `develop`. Commits directos a `master` o `main` sin ramas `feature/` ni PRs anulan el **100% de la nota de Git (15%)**.

9 Metodología Oficial de Costos de Software del Ing. Eduardo Nieves

Uno de los errores más destructivos en la industria es presupuestar proyectos por "tarifa plana" sin discriminar costos fijos ni depreciación. Las células deben aplicar la **Norma de 7 Pasos** para saber exactamente cuánto vale el software construido:

```
FÓRMULA MAESTRA DEL COSTO TOTAL CONSOLIDADO (PUNTO DE EQUILIBRIO TÉCNICO):
CTC = C_dir + C_op + C_labor + C_inv + C_desgaste
Base Universal de Productividad: 176 Horas Mensuales (22 Días Laborales × 8 Horas/Día) • Anual: 2.112 Horas
```

Paso Técnico	Concepto y Elementos Auditados	Fórmula Matemática Aplicada	Criterio de Control
Paso 1: Insumos Digitales Directos	Servicios cloud exclusivos para el cliente (tokens OpenAI, DB gestionada, hosting, dominio, SSL).	`C_dir = Σ Insumos Reales`	Facturas o cotizaciones reales de proveedores.
Paso 2: Costos Operativos Fijos	Gastos fijos de infraestructura del taller/estudio (internet fibra, electricidad, repositorios, coworking).	`Tarifa_Op = Gastos_Fijos / 176 h` `C_op = Tarifa_Op × Horas_Tickets`	Prorrateo matemático estricto sobre las 176 horas mes.
Paso 3: Mano de Obra Técnica	Salario base del desarrollador (Frontend, Backend, Full-Stack) liquidado por hora invertida.	`Tarifa_Mo = Sueldo_Mes / 176 h` `C_labor = Tarifa_Mo × Horas_Tickets`	**Debe coincidir 1:1 con las horas de la columna Terminado.**
Paso 4: Inversión Anual en Hardware	Amortización de computadoras de desarrollo (2-3 años de vida útil), cursos y certificaciones técnicas.	`Tarifa_Inv = Presupuesto_Año / 2.112 h` `C_inv = Tarifa_Inv × Horas_Tickets`	Fondo de reposición técnica de equipamiento.
Paso 5: Desgaste & Contingencia (20%)	Factor técnico ineludible que cubre bugs inesperados, code reviews, staging, retroalimentación y refactorización.	`C_desgaste = (C_dir + C_op + C_labor + C_inv) × 0.20`	**20% estricto** sobre el subtotal acumulado de Pasos 1 a 4.
Paso 6: Costo Total Consolidado	Costo real de manufactura tecnológica del software entregado.	`CTC = C_dir + C_op + C_labor + C_inv + C_desgaste`	Punto de Equilibrio (0% Ganancia).
Paso 7: Banda Absoluta de Precios	Fijación de los 3 niveles de precio comercial blindado: • **Piso Mínimo (+30%):** `CTC × 1.30` (Límite infranqueable). • **Precio Estándar (+40% a +50%):** `CTC × 1.40 a 1.50`. • **Techo Enterprise (+60% a +100%):** `CTC × 1.60 a 2.00`.	`Precio_Venta = CTC × (1 + %Margen)`	Si un cliente pide rebaja, se reduce alcance (Scope), nunca el margen.

10 La Matriz Espejo de Conciliación: "Todo Debe Cuadrar"

El informe debe incluir la siguiente tabla resumen demostrando que la suma de horas del tablero Kanban es idéntica al tiempo liquidado en el modelo de costos:

Ticket ID	Descripción del Feature / Módulo (Célula de 5 Devs)	Rama GitFlow / PR	Horas Reales	Tarifa MO ($)
`TASK-101`	Diseño DDL, Migraciones y Conexión DB (Git Lead)	`feature/101-ddl-db` (PR #02)	**6.0 horas**	6.0 h × $6.82 = $40.92
`TASK-102`	Controladores y Rutas API REST CRUD (Dev 2)	`feature/102-api-crud` (PR #05)	**11.0 horas**	11.0 h × $6.82 = $75.02
`TASK-103`	Vistas Frontend, Consumo Fetch y Feedback (Dev 3)	`feature/103-vistas-ui` (PR #08)	**11.5 horas**	11.5 h × $6.82 = $78.43
`TASK-104`	Validaciones, Manejo de Errores y DTOs (Dev 4)	`feature/104-validaciones` (PR #11)	**9.0 horas**	9.0 h × $6.82 = $61.38
`TASK-105`	Pruebas de Integración y Despliegue Local (Dev 5)	`feature/105-pruebas` (PR #14)	**7.5 horas**	7.5 h × $6.82 = $51.15
	TOTAL HORAS REGISTRADAS EN TABLERO (5 INTEGRANTES):		**45.0 HORAS**	**$306.90 USD**

Alerta Crítica de Auditoría Contable — "Todo Debe Cuadrar":
Si la sumatoria de las tarjetas de los 5 integrantes en *Terminado* arroja **45.0 horas** y el informe liquida sobre **55 horas** o **35 horas**, la entrega se considera **fraudulenta por falta de trazabilidad** y se anula la puntuación del informe de costos.

11 Rúbrica de Evaluación Analítica Oficial del Segundo Corte

Dimensión Evaluada	Sobresaliente (100%)	Competente (75%)	Básico / Insuf. (≤50%)	Ponderación
1. Frontend Dinámico & Manejo de Estados	Consume endpoints vía HTTP en tiempo real. Maneja perfectamente los 3 estados (Loading, Success, Error). Formularios con validación en cliente.	Consume API correctamente pero carece de feedback visual de carga o notificaciones de error adecuadas.	Datos quemados o simulados en JS/HTML. Vistas no reactivas o rotas al fallar la red.	**20%**
2. Backend RESTful & Persistencia DB	Arquitectura en capas limpia. Endpoints CRUD completos y semánticos. Conexión activa a base de datos relacional con claves foráneas.	Endpoints funcionales pero arquitectura acoplada en un solo archivo. Persistencia correcta sin migraciones ordenadas.	Falta de operaciones CRUD clave, caída del servidor ante excepciones o base de datos no conectada.	**25%**
3. Flujo GitFlow & Tablero Kanban	Correspondencia estricta de 4 columnas (Por hacer, Progreso [feature/<ticket>-<titulo>], Revisión [PRs hacia develop], Terminado [merged develop / master]). Nomenclatura obligatoria al 100%.	Tablero actualizado pero se observan ramas huérfanas o PRs cerrados sin revisión adecuada.	Commits directos a main/develop sin ramas ni PRs. Tablero desactualizado o inexistente.	**15%**
4. Trazabilidad de Horas por Tarjeta	Cada card de la columna Terminado detalla descripción técnica, responsable asignado y horas reales coherentes con el esfuerzo.	La mayoría de los tickets registran horas, pero algunos presentan descripciones vagas o falta de justificación en desvíos.	Tarjetas sin registro de horas o estimaciones ridículas e inconsistentes con el software.	**10%**
5. Informe de Valoración de Costos	Aplica con exactitud los 7 Pasos (Directos, Fijos/176h, MO, Inversión/2112h, 20% Desgaste, CTC y Banda Absoluta). Conciliación perfecta de horas.	Aplica la fórmula matemática pero comete errores menores de prorrateo o falta el cálculo de la Banda Absoluta.	No aplica la metodología oficial. Las horas del informe no cuadran con las del tablero Kanban.	**20%**
6. Defensa Ágil: Flujo del Módulo y Costos (5 a 7 min)	Demostración ejecutiva de 5 a 7 min. Muestran con soltura el flujo del módulo (UI reactiva ➔ API ➔ DB) y defienden con exactitud cuánto cuesta el módulo entregado según las horas reales del ticket y los 7 pasos.	Muestran el flujo funcional del módulo pero dudan o presentan imprecisiones al justificar el costo económico del desarrollo entregado.	El equipo no logra mostrar el flujo funcional en funcionamiento o no sabe justificar el valor económico y las horas del módulo.	**10%**
			TOTAL GENERAL EVALUACIÓN V:	**100%**

12 Escala de Penalizaciones Directas en la Nota Final

-20% • Datos Quemados
Uso de tablas o datos estáticos en el cliente que no provengan de consultas asíncronas al backend.

-15% • Violación de GitFlow
Commits directos en `main` / `master`, repositorio no clonable o discrepancia entre lo mostrado en clase y lo subido a GitHub.

-20% • Descuadre Contable
Disparidad entre las horas de los tickets terminados y las horas calculadas en el informe económico.

13 Ficha de Identificación de la Célula de Desarrollo

Nombre del Sistema / Proyecto: _____
Célula / N° Equipo (5 Devs): _____ • **Sección:** _____
Modalidad: [ ] Monorepo [ ] Repositorios Independientes
URL Repo Frontend / Monorepo: _____
URL Repo Backend (si aplica): _____
URL Tablero Ágil (GitHub / Trello / Asana / Monday / Jira): _____
Visibilidad Repo: [ ] Público [ ] Privado (Invitación enviada a `@zedmous`)
Motor de Base de Datos: _____ • **Stack Backend:** _____
Framework / Stack Frontend: _____

14 Integrantes de la Célula (Desarrollo Full-Stack) y Trazabilidad Individual de Horas

Nota Pedagógica: Células de 5 integrantes (o 4 según nómina oficial). Todos desarrollan en modalidad Full-Stack. Un estudiante coordina el repositorio y los merges como Líder Técnico (Git Lead).

Apellidos y Nombres	Cédula Identidad	Rol Técnico en la Célula	Horas en Tablero	Firma de Conformidad
1.		Líder Técnico / Git Lead (Full-Stack)		
2.		Desarrollador Full-Stack (Módulo: _____)		
3.		Desarrollador Full-Stack (Módulo: _____)		
4.		Desarrollador Full-Stack (Módulo: _____)		
5.		Desarrollador Full-Stack (Módulo: _____)		

15 Lista de Verificación Técnica en Vivo (Checklist del Docente)

Defensa Ágil en Vivo (5 a 7 min): Flujo y Costos

[ ] **Flujo del Módulo en Vivo:** Interacción comprobada (UI ➔ API ➔ BD).
[ ] **Costo del Módulo Defendido:** Justificación económica según 7 pasos.
[ ] **Cero Simulación:** Las consultas consumen API HTTP real con DB activa.
[ ] **Tiempo Respetado:** Demostración concisa y ejecutiva en 5 a 7 minutos.

Auditoría de Git, Kanban y Costos

[ ] **Repositorio Clonable & Auditado:** Código verificado por el docente; coincide 1:1 con la demo.
[ ] **GitFlow Cumplido:** Ramas master, develop y feature/<num>-<titulo>.
[ ] **Kanban Actualizado:** Tarjetas en columna Terminado con horas.
[ ] **Cuadre Matemático:** Total de horas de tickets = Horas Informe.

16 Veredicto y Liquidación Sumativa de Calificaciones

Dimensión Evaluada	Ponderación (%)	Calificación (%)	Observaciones Técnicas del Docente
1. Frontend Dinámico & Manejo de Estados	**20%**		
2. Backend RESTful & Persistencia en DB	**25%**		
3. Flujo GitFlow & Tablero Kanban	**15%**		
4. Trazabilidad de Horas por Tarjeta	**10%**		
5. Informe de Valoración de Costos TI	**20%**		
6. Defensa Ágil: Flujo del Módulo y Costos (5 a 7 min)	**10%**		
CALIFICACIÓN DEFINITIVA EVALUACIÓN V:	**100%**	_____ **%**	

Prof. Eduardo Nieves
Docente Titular de Cátedra ADS-433

Líder de Célula de Desarrollo
Firma y Cédula en Conformidad